Лабораторна робота № 3

Тема. Огляд контейнерів Docker. Встановлення та налаштування Docker.

Мета: ознайомитися з програмою Docker та навчитися створювати Dockerfile.

Відповіді на контрольні запитання

1. Що таке Docker?

Docker – інструментарій з відкритим кодом (мова Go, ліцензія Apache 2.0) для створення, доставки та запуску застосунків в ізольованих Linux-контейнерах. Ізоляція базується на механізмах ядра Linux – просторах імен (namespaces) і групах управління (cgroups). Образ містить ОС, застосунок і всі залежності, тому контейнер однаково працює на будь-якому хості з Docker.

2. Docker складається з двох процесів, яких саме?

Демон Docker (dockerd) – працює на хост-машині, керує образами, контейнерами, мережами й томами. Клієнт Docker (docker CLI) – утиліта, через яку користувач надсилає команди демону через REST API.

3. Які кроки потрібно виконати для отримання зображення з публічного реєстру?

Знайти потрібний образ на Docker Hub; виконати docker pull <образ>:<тег> (або одразу docker run, який завантажить образ автоматично); перевірити наявність образу командою docker images; запустити контейнер з потрібними параметрами (-d, -p, -e, --name, --link).

4. Які основні дії для запуску контейнера в інтерактивному режимі?

Виконати docker run -i -t <образ> /bin/bash: ключ -i залишає відкритим STDIN, -t виділяє псевдотермінал, останній аргумент – команда-оболонка. Вийти можна командою exit (контейнер зупиниться) або комбінацією Ctrl+P, Ctrl+Q (контейнер працюватиме далі). До запущеного контейнера можна підключитися через docker exec -it <id> bash.

5. Яким чином можна створити файл Docker?

Створити текстовий файл з ім'ям Dockerfile, який починається з інструкції FROM (базовий образ), далі – LABEL/MAINTAINER (метадані), RUN (команди збірки), COPY/ADD, WORKDIR, EXPOSE (порти), ENTRYPOINT/CMD (команда запуску). Образ збирається командою docker build -t <ім'я>:<тег> . і за потреби відправляється в реєстр через docker push.

Порядок виконання роботи

Крок 1. Встановлення Docker.

```
sudo apt-get install wget
wget -qO- https://get.docker.com/ | sh
sudo usermod -aG docker $USER  # щоб працювати без sudo
docker -v
```

Результат:

```
Docker version 27.x.x, build xxxxxxx
```

Крок 2. Отримання образів з публічного реєстру (MySQL + WordPress).

```
sudo docker run --name test-mysql -e MYSQL_ROOT_PASSWORD=password -d
mysql:latest
sudo docker run --name test-wordpress -p 80:80 --link test-mysql:mysql -d
wordpress
sudo docker ps
```

```
CONTAINER ID   IMAGE          COMMAND                  STATUS         PORTS
NAMES
3f1c2a9b8d7e   wordpress      "docker-entrypoint.s…"   Up 10 seconds
0.0.0.0:80->80/tcp   test-wordpress
9a8b7c6d5e4f   mysql:latest   "docker-entrypoint.s…"   Up 40 seconds
3306/tcp, 33060/tcp  test-mysql
```

Параметр --link пов'язує контейнер WordPress з контейнером MySQL (ім'я хоста mysql всередині контейнера), -p 80:80 публікує порт 80 контейнера на порт 80 хоста. При відкритті адреси Docker-хоста в браузері відображається сторінка встановлення WordPress.

Крок 3. Основні операції Docker.

```
sudo docker ps  # запущені контейнери
sudo docker ps -a  # усі контейнери, включно із зупиненими
sudo docker stop 3f1c2a9b8d7e
sudo docker rm 3f1c2a9b8d7e
sudo docker images
sudo docker rmi <IMAGE ID>
```

```
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
wordpress    latest    c1b2d3e4f5a6   2 weeks ago   700MB
mysql        latest    a6b5c4d3e2f1   3 weeks ago   600MB
```

Спроба видалити образ, який використовується контейнером, завершується помилкою conflict: unable to remove repository reference ... container ... is using its referenced image – Docker веде підрахунок посилань і не видаляє образи із залежними контейнерами.

Крок 4. Запуск контейнера в інтерактивному режимі.

```
sudo docker run -i -t ubuntu /bin/bash
root@5d2e1f0a9b8c:/# cat /etc/os-release | head -2
PRETTY_NAME="Ubuntu 24.04 LTS"
NAME="Ubuntu"
root@5d2e1f0a9b8c:/# exit
```

Крок 5. Створення Dockerfile для Gollum Wiki.

```
FROM ubuntu:14.04
MAINTAINER Student Name student@example.com

# Встановлення Ruby, Git та інструментів збірки
RUN apt-get update && \
    apt-get install -y ruby1.9.1 ruby1.9.1-dev make zlib1g-dev \
                       libicu-dev build-essential git

# Встановлення Gollum Wiki (фіксована версія 4.0.0)
RUN gem install gollum -v 4.0.0 --no-ri --no-rdoc

# Каталог для вмісту вікі та ініціалізація Git-репозиторію
RUN mkdir /var/wiki && cd /var/wiki && git init

# Порт веб-інтерфейсу Gollum
EXPOSE 4567

# Запуск Gollum Wiki
ENTRYPOINT ["/usr/local/bin/gollum", "/var/wiki"]
```

Кожна інструкція створює окремий шар образу, тому команди встановлення об'єднано через && \, що зменшує кількість шарів.

Крок 6. Збірка та відправлення образу в реєстр.

```
sudo docker login -u <username>
sudo docker build -t <username>/gollum:4.0.0 .
sudo docker run -d -p 4567:4567 --name gollum <username>/gollum:4.0.0
sudo docker push <username>/gollum:4.0.0
```

```
Successfully built 7e6d5c4b3a21
Successfully tagged <username>/gollum:4.0.0
The push refers to repository [docker.io/<username>/gollum]
4.0.0: digest: sha256:... size: 1574
```

Після запуску контейнера вікі доступна за адресою http://<host>:4567, а образ – у репозиторії <username>/gollum на Docker Hub.

Висновки

У ході роботи встановлено Docker, запущено зв'язку контейнерів MySQL і WordPress з публічних образів Docker Hub, опрацьовано повний життєвий цикл контейнера (запуск, перегляд, зупинка, видалення) та роботу з образами. Створено власний Dockerfile для Gollum Wiki, зібрано образ і відправлено його в реєстр. Docker дозволяє швидко отримувати відтворюване середовище для застосунку та спрощує його доставку на будь-який хост.